IEXTEN Flag Explanation

IEXTEN meaning

In the context of terminal input processing, `IEXTEN` is a flag used in terminal control settings (such as with the `termios` structure in Unix-like operating systems). It stands for "Input EXTension" and is part of the `c_lflag` field in the terminal attributes.
When `IEXTEN` is enabled, it allows certain implementation-defined input processing features to be active. These features can include things like special character sequences for signal generation, output processing, and extended line discipline functions.
For example, enabling `IEXTEN` might allow the use of the `^V` (literal next) character, which can be used to input the next character literally, bypassing its normal interpretation by the terminal.
Disabling `IEXTEN` can turn off these extra features, leading to more raw input handling, where characters are passed through with less modification or interpretation.

ways to handle non-printing of ctrl characters, except

To handle the non-printing of control characters (like Ctrl+C, Ctrl+D) except for specific ones (such as `Ctrl+V` for literal next), you can configure terminal settings to suppress the output of control characters while allowing certain exceptions. Here's how you can do it:

1. Modify `termios` Flags in C

You can use the `termios` structure in Unix-like systems to control how input is handled. To disable control character printing but allow certain exceptions (like `Ctrl+V`), you would modify the `c_lflag` and `c_iflag` fields.

Steps:

1. Retrieve the current terminal attributes using `tcgetattr()`.
2. Modify the `termios` settings to disable features you don't want (like the echo of control characters).
3. Apply the changes using `tcsetattr()`.

Here's an example of how to disable printing of control characters, except for `Ctrl+V`:

c

```c
#include <termios.h>
#include <unistd.h>
#include <stdio.h>

void disable_ctrl_characters() {
    struct termios term;
    tcgetattr(STDIN_FILENO, &term);

    // Disable control characters echoing by turning off ECHOCTL (may vary by system)
    term.c_lflag &= ~ECHOCTL;

    // Optionally disable other input processing like signal generation with ISIG, if
needed
    // term.c_lflag &= ~ISIG; // Disable signal generation (Ctrl+C, Ctrl+Z, etc.)

    // Apply changes
    tcsetattr(STDIN_FILENO, TCSANOW, &term);
}

void enable_ctrl_characters() {
    struct termios term;
    tcgetattr(STDIN_FILENO, &term);

    // Enable control characters echoing
    term.c_lflag |= ECHOCTL;

    // Apply changes
    tcsetattr(STDIN_FILENO, TCSANOW, &term);
}

int main() {
    disable_ctrl_characters();
    printf("Control characters won't be printed except for Ctrl+V\n");

    // Example of how you can read input
    char buf[100];
    fgets(buf, sizeof(buf), stdin);

    enable_ctrl_characters();  // Restore default behavior
    printf("Input: %s\n", buf);
    return 0;
}
```

Explanation:

- `ECHOCTL`: This flag, if set, echoes control characters as `^C`, `^

Give equivalent alternatives to ECHOCTL

If your system or terminal environment doesn't support the `ECHOCTL` flag for controlling the display of control characters, you can use alternative approaches or flags to achieve similar behavior. Here are some equivalent alternatives or workarounds:

1. Disabling `ICANON` Mode for Raw Input

- Instead of using `ECHOCTL`, you can disable canonical mode (`ICANON`). In this mode, control characters won't be processed or echoed, and the input will be read byte-by-byte, which effectively suppresses the printing of control characters.

Example:

```c
term.c_lflag &= ~ICANON;  // Disable canonical mode
```

- In raw mode, control characters like `Ctrl+C` won't be echoed, but the terminal will stop processing input line-by-line, so you'll need to handle input on a per-character basis.

2. Disabling `ECHO` (Echo Control Characters Manually)

- Disabling the `ECHO` flag completely can prevent all characters, including control characters, from being printed to the terminal. You can then manually handle which characters are echoed back to the terminal by implementing a custom input handler.

Example:

```c
term.c_lflag &= ~ECHO;  // Disable all echoing, including control characters
```

- You can then use a custom routine to echo back only the characters you want to display, like regular printable characters or a specific control sequence such as `Ctrl+V`.

3. Modifying Input Flags with `ISTRIP` or `IGNBRK`

- If you want to strip the eighth bit from input bytes and suppress certain control sequences, you can use `ISTRIP`. This will ensure that control characters beyond ASCII 127 are handled differently.

Example:

```c
term.c_iflag |= ISTRIP;  // Strip the 8th bit of each input byte
```

- This is less directly related to control characters like `Ctrl+C`, but it can be useful in specific scenarios where you need to manipulate the input format.
- Alternatively, `IGNBRK` can be used to ignore break conditions (another form of control input):

```c
term.c_iflag |= IGNBRK;  // Ignore break conditions
```

4. Setting `VSTOP` and `VSTART` for XON/XOFF Flow Control

- You can control terminal flow using **`VSTOP`** and **`VSTART`**, which are typically mapped to **`Ctrl+S`** and **`Ctrl+Q`** for stopping and starting output. This can effectively handle certain control characters without needing **`ECHOCTL`**.

Example:

```c
tcgetattr(STDIN_FILENO, &term);
term.c_cc[VSTOP] = 0;  // Disable Ctrl+S (stop output)
term.c_cc[VSTART] = 0;  // Disable Ctrl+Q (start output)
```

5. Using `stty` for Manual Terminal Control

- In some cases, you can use the **`stty`** command to set or unset terminal behavior directly from the command line instead of in code:

```bash
stty -echoctl  # Disable control character echo
```

- This provides a quick way to manage terminal settings from the shell for temporary changes.

6. Switch to Raw Mode (`cfmakeraw`)

- To bypass most terminal input/output processing (including control characters), you can use **`cfmakeraw()`**. This sets the terminal to raw mode, where no special character processing occurs.

Example:

```c
cfmakeraw(&term);  // Set terminal to raw mode (disables all special input processing)
```

- In raw mode, control characters are not interpreted by the terminal and won't be echoed or processed.

By using any of these alternatives, you can control how control characters are handled or echoed without relying on the **`ECHOCTL`** flag.

stty -echoctl # Disable control character echo
how to reenable

To re-enable control character echo after using `stty -echoctl`, you can use the `stty echoctl` command.
Here are both commands for disabling and re-enabling the echo of control characters:

Disable control character echo:

```bash
stty -echoctl
```

Re-enable control character echo:

```bash
stty echoctl
```

The `echoctl` flag controls whether control characters like `Ctrl+C`, `Ctrl+Z`, etc., are displayed as `^C`, `^Z`, etc., in the terminal.

I have to disable Ctrl+C printing, Ctrl+D printing and Ctrl+Z printing, all others must be printied

To disable the printing of only specific control characters like `Ctrl+C` (SIGINT), `Ctrl+D` (EOF), and `Ctrl+Z` (SIGTSTP) while allowing other control characters (like `Ctrl+V` for literal next) to be printed, you need a combination of terminal control flag settings and possibly some custom handling of signals in your application.
Here's how you can achieve this in a Unix-like environment:

Approach:

1. **Modify Terminal Settings**: Use the `termios` structure to control which control characters are processed and echoed by the terminal.
2. **Signal Handling**: For `Ctrl+C` (SIGINT) and `Ctrl+Z` (SIGTSTP), you can trap these signals to prevent the default behavior (which includes printing) and customize how they are handled.

Code Example (C):

The following code demonstrates how to disable printing of `Ctrl+C`, `Ctrl+D`, and `Ctrl+Z`, while allowing other control characters to be printed:

```c
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <termios.h>
#include <unistd.h>
#include <signal.h>

struct termios original_termios;

void restore_terminal_settings() {
    // Restore original terminal settings before exiting
    tcsetattr(STDIN_FILENO, TCSANOW, &original_termios);
}

void disable_ctrl_characters() {
    struct termios term;

    // Get current terminal settings
    tcgetattr(STDIN_FILENO, &term);

    // Disable echoing of control characters like Ctrl+C, Ctrl+D, and Ctrl+Z
    // by trapping the corresponding signals and handling EOF (Ctrl+D) manually
    term.c_lflag &= ~ECHOCTL; // Disable control character echo

    // Disable signals (Ctrl+C and Ctrl+Z) from being processed by terminal
    term.c_lflag &= ~ISIG;

    // Apply changes
    tcsetattr(STDIN_FILENO, TCSANOW, &term);
}

void handle_signal(int sig) {
    if (sig == SIGINT) {
        printf("\nCaught SIGINT (Ctrl+C), but not printing it.\n");
    } else if (sig == SIGTSTP) {
        printf("\nCaught SIGTSTP (Ctrl+Z), but not printing it.\n");
    }
}

int main() {
    // Store original terminal settings
    tcgetattr(STDIN_FILENO, &original_termios);
    atexit(restore_terminal_settings);  // Ensure terminal settings are restored on exit

    // Set up signal handling for Ctrl+C and Ctrl+Z
    signal(SIGINT, handle_signal);
    signal(SIGTSTP, handle_signal);

    // Disable printing for specific control characters
    disable_ctrl_characters();

    // Input loop
    printf("Type something (Ctrl+C, Ctrl+D, and Ctrl+Z won't print):\n");
    char c;
    while (1) {
        c = getchar();
        if (c == 4) { // Ctrl+D (EOF)
            printf("\nCaught EOF (Ctrl+D), but not printing it.\n");
            break;
        }
        if (c != EOF) {
            putchar(c);  // Print other characters
        }
    }

    return 0;
}
```

Explanation:

- **Terminal Settings**:
  - `ECHOCTL` is disabled to stop the echoing of control characters in the terminal.

    - `ISIG` is disabled to prevent the terminal from automatically processing signals like `SIGINT` (Ctrl+C) and `SIGTSTP` (Ctrl+Z). This allows us to handle these signals manually without letting the terminal print them.
- **Signal Handling**:
    - The `signal()` function is used to catch `SIGINT` (Ctrl+C) and `SIGTSTP` (Ctrl+Z). In the `handle_signal()` function, we print custom messages instead of the usual `^C` or `^Z`.
- **Ctrl+D**:
    - For `Ctrl+D`, which indicates the end of input (EOF), we handle it directly by checking if the input character is 4 (ASCII for `Ctrl+D`). We can choose not to print it and handle EOF logic.
- **Restoring Terminal Settings**:
    - The original terminal settings are restored before the program exits to ensure that the terminal behaves normally after the program ends.

Summary of Key Points:

- `Ctrl+C`, `Ctrl+D`, and `Ctrl+Z` will not be printed.
- Other control characters (like `Ctrl+V`) will continue to be printed.
- You can customize the handling of specific control characters by adding more conditions or signal handlers.

This approach gives you fine-grained control over which control characters get printed and how others are handled.

I only want non-printing of ctrl+ characters, arrows must continune ot the vm

To achieve the behavior where `Ctrl+C`, `Ctrl+D`, and `Ctrl+Z` do not print to the terminal, but all other input (including arrow keys, regular characters, and special sequences like `Ctrl+V`) is handled as normal, you can adjust terminal settings specifically to suppress the display of those `Ctrl+` sequences without affecting other input.
Here's a refined solution that meets your requirements:

Approach:

1. **Suppress Printing of Specific Control Characters**: We'll disable printing of `Ctrl+C`, `Ctrl+D`, and `Ctrl+Z`, but leave other input handling unchanged.
2. **Keep Arrow Keys Working Normally**: Arrow keys will send escape sequences to the terminal, which should pass through unchanged.

Code Example (C):

This program disables the printing of `Ctrl+C`, `Ctrl+D`, and `Ctrl+Z`, while allowing everything else, including arrow keys and other control characters like `Ctrl+V`:

c

```c
#include <stdio.h>
#include <stdlib.h>
#include <termios.h>
#include <unistd.h>
#include <signal.h>

struct termios original_termios;

// Restore original terminal settings on exit
void restore_terminal_settings() {
    tcsetattr(STDIN_FILENO, TCSANOW, &original_termios);
}

// Configure terminal to suppress specific control characters
void configure_terminal() {
    struct termios term;

    // Get current terminal settings
    tcgetattr(STDIN_FILENO, &term);

    // Disable echoing of control characters like Ctrl+C, Ctrl+D, Ctrl+Z
    term.c_lflag &= ~ECHOCTL;  // Disable printing of control characters
    term.c_lflag &= ~ISIG;     // Disable signal handling for Ctrl+C and Ctrl+Z

    // Apply changes
    tcsetattr(STDIN_FILENO, TCSANOW, &term);
}

// Signal handler for Ctrl+C and Ctrl+Z
void handle_signal(int sig) {
    // Suppress printing of Ctrl+C or Ctrl+Z, but handle signals
    if (sig == SIGINT) {
        printf("\nCaught SIGINT (Ctrl+C), but not printing it.\n");
    } else if (sig == SIGTSTP) {
        printf("\nCaught SIGTSTP (Ctrl+Z), but not printing it.\n");
    }
}

int main() {
    // Save original terminal settings and restore on exit
    tcgetattr(STDIN_FILENO, &original_termios);
    atexit(restore_terminal_settings);

    // Setup signal handling for Ctrl+C (SIGINT) and Ctrl+Z (SIGTSTP)
    signal(SIGINT, handle_signal);
    signal(SIGTSTP, handle_signal);

    // Configure the terminal
    configure_terminal();

    // Input loop to demonstrate the behavior
    printf("Type something (Ctrl+C, Ctrl+D, and Ctrl+Z won't print, arrows will
work):\n");
    char c;
    while (1) {
        c = getchar();
        if (c == 4) {  // Ctrl+D (EOF)
            printf("\nCaught EOF (Ctrl+D), but not printing it.\n");
            break;
        }
        if (c != EOF) {
            putchar(c);  // Echo normal input back
        }
    }

    return 0;
}
```

Key Details:

1. **Disabling `ECHOCTL`**:
   - This disables the printing of control characters like `Ctrl+C` (`^C`), `Ctrl+Z` (`^Z`), etc., to prevent them from being shown.
2. **Disabling `ISIG`**:
   - This prevents the terminal from automatically handling signals like `SIGINT` (Ctrl+C) and `SIGTSTP` (Ctrl+Z), so we can catch these signals manually and handle them without printing.
3. **Handling Ctrl+D (EOF)**:
   - When `Ctrl+D` is pressed, we capture the EOF condition and prevent it from being printed. We break the loop when EOF is detected but prevent it from being displayed.
4. **Arrow Keys**:
   - Arrow keys and other special sequences send escape sequences (starting with `\x1B`). These escape sequences are handled by the terminal's default behavior, which is not affected by the changes made to suppress control characters. Thus, the arrow keys continue to work as normal and are passed through to the terminal.

Testing the Behavior:

1. **Ctrl+C**: When you press `Ctrl+C`, it will trigger `SIGINT`, but it won't display `^C`. Instead, it will print the custom message: "Caught SIGINT (Ctrl+C), but not printing it."
2. **Ctrl+D**: When you press `Ctrl+D`, it will not display `^D` and will stop the loop after printing: "Caught EOF (Ctrl+D), but not printing it."
3. **Ctrl+Z**: When you press `Ctrl+Z`, it will trigger `SIGTSTP`, but it won't display `^Z`. Instead, it will print: "Caught SIGTSTP (Ctrl+Z), but not printing it."
4. **Arrow Keys**: Arrow keys will continue to function normally (e.g., for moving the cursor in a terminal-based text editor or similar environment).

Summary:

This approach achieves the following:

- **Disables printing of specific control characters** (`Ctrl+C`, `Ctrl+D`, `Ctrl+Z`).
- **Keeps arrow keys and other input unchanged** and handled normally by the terminal.
- **Allows custom signal handling** for specific control characters without terminal-side printing.

how to restore stdin and stdout if I lose the original descriptors

If you lose the original file descriptors for `stdin` (file descriptor 0) or `stdout` (file descriptor 1) and need to restore them, there are several methods to handle this situation. Below are a few approaches to restore them, depending on whether you have saved the original file descriptors beforehand or not.

1. Save and Restore File Descriptors

A common approach is to **save the original descriptors** before modifying them (e.g., before redirecting). This allows you to restore them later.

Code Example (C):

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    // Save the original file descriptors for stdin and stdout
    int saved_stdin = dup(STDIN_FILENO);
    int saved_stdout = dup(STDOUT_FILENO);

    // Redirect stdin and stdout (for example, to /dev/null)
    int dev_null = open("/dev/null", O_RDWR);
    dup2(dev_null, STDIN_FILENO);
    dup2(dev_null, STDOUT_FILENO);

    // Use redirected stdin and stdout (this won't print anything)
    printf("This goes to /dev/null\n");

    // Restore the original stdin and stdout
    dup2(saved_stdin, STDIN_FILENO);
    dup2(saved_stdout, STDOUT_FILENO);

    // Close the saved descriptors and /dev/null
    close(saved_stdin);
    close(saved_stdout);
    close(dev_null);

    // Now, stdin and stdout are restored
    printf("This goes to the original stdout\n");

    return 0;
}
```

Explanation:

- **Save original descriptors**: We use `dup()` to make a copy of the original `stdin` and `stdout` descriptors.
- **Redirect to `/dev/null`**: We redirect `stdin` and `stdout` to `/dev/null` as an example.
- **Restore**: After the redirection, we use `dup2()` to restore the original `stdin` and `stdout` from the saved descriptors.

2. Reopen `stdin` and `stdout` from `/dev/tty`

If you've lost the original descriptors and didn't save them, you can restore the terminal's input and output by reopening `/dev/tty`, which refers to the controlling terminal.

Code Example (C):

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
```

```
int main() {
    // Lose original stdin and stdout by redirecting them
    int dev_null = open("/dev/null", O_RDWR);
    dup2(dev_null, STDIN_FILENO);
    dup2(dev_null, STDOUT_FILENO);

    // Reopen stdin and stdout from /dev/tty
    int tty_in = open("/dev/tty", O_RDONLY);
    int tty_out = open("/dev/tty", O_WRONLY);
    dup2(tty_in, STDIN_FILENO);
    dup2(tty_out, STDOUT_FILENO);

    // Close the /dev/tty file descriptors
    close(tty_in);
    close(tty_out);
    close(dev_null);

    // Now stdin and stdout are restored
    printf("This is printed to the terminal again!\n");

    return 0;
}
```

Explanation:

- **Reopen `/dev/tty`**: We open `/dev/tty` as a file to restore both input (`O_RDONLY`) and output (`O_WRONLY`). `/dev/tty` always refers to the controlling terminal for a process, so it works well for restoring standard input and output.
- **Restore descriptors**: Use `dup2()` to redirect the `/dev/tty` file descriptors to `stdin` and `stdout`.

3. Use `/proc/self/fd/0` and `/proc/self/fd/1` on Linux

If you're working on a Linux system, another option is to reopen `stdin` and `stdout` using `/proc/self/fd/0` and `/proc/self/fd/1`. These refer to the file descriptors of the running process, and you can use them to reopen the original `stdin` and `stdout`.

Code Example (Linux only):

c

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    // Lose original stdin and stdout by redirecting them
    int dev_null = open("/dev/null", O_RDWR);
    dup2(dev_null, STDIN_FILENO);
    dup2(dev_null, STDOUT_FILENO);

    // Reopen stdin and stdout from /proc/self/fd/0 and /proc/self/fd/1
    int stdin_fd = open("/proc/self/fd/0", O_RDONLY);
    int stdout_fd = open("/proc/self/fd/1", O_WRONLY);
    dup2(stdin_fd, STDIN_FILENO);
    dup2(stdout_fd, STDOUT_FILENO);

    // Close the /dev/null and proc descriptors
    close(stdin_fd);
    close(stdout_fd);
    close(dev_null);
```

```
    // Now stdin and stdout are restored
    printf("This is printed to the terminal again!\n");

    return 0;
}
```

Explanation:

- **Reopen from `/proc/self/fd/`**: On Linux, `/proc/self/fd/0` and `/proc/self/fd/1` refer to the current process's `stdin` and `stdout`. By opening these paths, you can restore the original input/output.
- **Restore using `dup2()`**: Once you have reopened the file descriptors, use `dup2()` to restore them to `stdin` and `stdout`.

Summary:

- **Best practice**: Always save the original descriptors using `dup()` before modifying them.
- **Reopen from `/dev/tty`**: If you've lost the original descriptors and didn't save them, reopening from `/dev/tty` can restore terminal input/output.
- **Linux-specific approach**: On Linux, you can reopen `stdin` and `stdout` using `/proc/self/fd/0` and `/proc/self/fd/1`.